

Channel Fracture: Three Instances of Cross-Boundary Silent Failure in Multi-Agent Orchestration Systems

Dexing Liu¹

Abstract

\newcommand{\vTotal}{40,020} \newcommand{\vCTRL}{30,012}
\newcommand{\vREAL}{10,008} \newcommand{\vTRIALSTHREE}{3,000}
\newcommand{\vTRIALSFOUR}{3,000} \newcommand{\vTRIALSFIVE}{3,006}
We report the discovery and systematic characterization of **channel fracture**—a class of architectural failure mode in multi-agent orchestration systems where information routed across agent, profile, or subsystem boundaries is silently blocked by invisible architectural constraints. Unlike command injection or output injection attacks, channel fracture is a **silent failure**: the source agent reports success, no error is raised, and yet the target never receives the intended information. We present **three empirically validated instances** of channel fracture discovered in a production Hermes Agent deployment: (1)~cron-delegated memory injection blocked by hardcoded `skip_memory=True` barriers in the scheduler layer; (2)~cross-profile skill namespace routing fractured by unbounded recursive directory traversal in the skill manager; and (3)~delivery confirmation fallback fracture (DCFF), where an unstable WebSocket delivery channel causes a well-intentioned loss-prevention safety net to produce message duplication. All three instances share a common architectural structure: a source agent dispatches information through a valid-seeming channel, an invisible constraint at the target boundary blocks or distorts delivery, and the system provides no error signal—the fracture is detectable only through receiver-side inspection. We propose **CADVP v1.1**, a 13-dimension cross-agent delivery verification protocol with a veto-level channel confirmation check (CC-0) that detects channel fractures at design time. We further extend CADVP with a **Three-Gate Quality System** (L1 Self-Verification, L2 Evidence Verification, L3 Cross-Review) for execution-level delivery assurance. Through three controlled experiment suites—cross-agent relay, exception recovery rollback, and concurrent conflict detection—totaling 30,012 controlled trials, we demonstrate zero failure rates under protocol protection versus 69--98% without. Real-world validation through 10,008 trials confirms quality elevation from 0.90 to 1.00 under full protocol protection. We

articulate three design principles: the inverse verification principle, the channel matching principle, and the PIP principal interest protection principle. Our findings establish channel fracture as a generalizable silent failure pattern in multi-agent systems with at least three distinct instances confirmed in production.

Keywords: multi-agent systems, persistent memory, cron scheduling, channel fracture, silent failure, CADVP, three-gate system, Hermes Agent, knowledge injection, PIP principal interest protection principle

Introduction

Motivation

The emergence of multi-agent AI teams---collections of specialized AI agents collaborating under orchestration frameworks---has created new requirements for inter-agent knowledge transfer. Unlike monolithic agents that maintain a single memory store, multi-agent architectures distribute memory across agent boundaries, requiring explicit mechanisms for one agent to inject information into another's persistent store~ [\[wang2024survey,zhang2024multi\]](#). Consider a practical scenario: a team administrator agent needs to ensure that a subordinate agent remembers a specific report format for future use. The administrator must write this knowledge into the subordinate's memory system so that it persists across sessions. This cross-agent memory injection is fundamental to coordinated multi-agent behavior.

The Problem

In production multi-agent deployments, knowledge injection often occurs through scheduled (cron) tasks---periodic or one-shot jobs dispatched by the orchestration layer. These tasks run in isolated execution contexts that differ from interactive agent sessions. We discovered that this isolation creates a **channel fracture**: a systematic, silent failure where the scheduled agent cannot access the memory tools necessary to complete its injection task, despite the task being correctly configured and dispatched. The fracture is particularly insidious because:

1. **It fails silently.** The cron job completes without error; the agent may report partial success.
2. **Writer-side verification passes.** From the sender's perspective, all steps appear correct.

3. **Only receiver-side inspection reveals the failure.** The target agent's memory remains empty.

Contributions

This paper makes the following contributions:

1. **Identification and characterization** of the channel fracture problem in multi-agent orchestration systems, with source-level root cause analysis across three empirically validated instances.
2. **Instance 1 (cron memory):** Experimental demonstration that cron-delegated memory injection fails completely due to `skip_memory=True` and conditional tool registration, while bypass channels (direct SQLite, self-write) succeed.
3. **Instance 2 (skill namespace):** Discovery of a cross-profile skill routing fracture where unbounded `rglob("SKILL.md")` recursion causes instructions to be routed to incorrect profile copies---a code-level channel fracture between profiles.
4. **Instance 3 (delivery confirmation):** Discovery of DCFF, a delivery-plane channel fracture where unreliable WebSocket ACK causes loss-prevention safety nets to trigger message duplication---demonstrating that channel fracture extends from data-plane to control-plane failures.
5. **CADVP v1.1**, a 13-dimension cross-agent delivery verification protocol with a veto-level channel confirmation check, validated across all three fracture types.
6. **Controlled experiments (30,012~total trials across three suites)** validating concurrent conflict detection, exception recovery, and cross-agent relay under the BCP protocol.
7. **Real-world validation (10,008~trials)** confirming quality improvements under protocol protection in production-like settings.
8. **The Three-Gate Quality System and PIP principal interest protection principle**---extending verification from channel-level to delivery-level and from technical to social-norm assurance.

Background

Hermes Agent Architecture

Hermes Agent~ [\[hermes2024\]](#) (Nous Research, 2024--2026) is an open-source, self-hosted AI agent framework supporting multi-agent orchestration through its profiles system. Each profile represents an independent agent with its own

configuration file (`config.yaml`), identity document (`SOUL.md`), persistent memory store (SQLite-backed `memory_store.db`), gateway process, and cron job definitions. Profiles operate as isolated agents that can be configured to collaborate. A typical deployment includes a primary profile and specialized subordinate profiles for different functional domains (development, business development, marketing, etc.).

Holographic Memory System

Hermes Agent implements persistent memory through holographic memory---a fact-based storage system backed by SQLite with FTS5 full-text search~[sqlite2024]. The memory system comprises:

- **fact_store**: A structured fact database supporting add, search, and retrieval through tool-call interfaces.
- **Memory banks**: Categorized collections of persistent facts.
- **Entity tracking**: Associations between facts and named entities.
- **Full-text search**: FTS5-indexed content for semantic retrieval.

The memory subsystem is initialized through a `_MemoryManager` component that registers memory provider tools into the agent's tool surface at initialization time. This registration is conditional:

```
if agent._memory_manager and \
    agent.tools is not None and (
        agent.enabled_toolsets is None or
        "memory" in agent.enabled_toolsets
    ):
    for _schema in agent._memory_manager
        .get_all_tool_schemas():
        _wrapped = {"type": "function",
                    "function": _schema}
        agent.tools.append(_wrapped)
```

If `_memory_manager` is not initialized (set to `None`), the `fact_store` tool is never registered and is unavailable to the agent at runtime.

Cron/Scheduler System

The Hermes Agent scheduler (`cron/scheduler.py`) provides periodic and one-shot task execution. Cron jobs run as independent agent sessions with specific constraints:

```
AIAgent(  
    ...  
    skip_memory=True, # Cron system prompts  
                      # would corrupt user  
                      # representations  
    platform="cron",  
    ...  
)
```

The `skip_memory=True` flag is a deliberate architectural decision to prevent cron system prompts from contaminating user-facing memory. However, this guard blocks all memory access---including legitimate, intentional writes.

Profile Isolation

Each profile configures `approvals.cron_mode`, which when set to `deny`, restricts potentially dangerous operations during cron execution. This provides a secondary isolation layer at the permission level.

The Channel Fracture Problem

Problem Definition

Definition 1 (Channel Fracture). Given a source agent *S*, a target agent *T*, and an injection channel *C* connecting them, a channel fracture occurs when *C* is valid at *S*'s execution context but invalid at *T*'s execution context, causing silent failure of knowledge delivery.

Experimental Design

We conducted experiments on a production Hermes Agent deployment with five active profiles:

\centering

Experimental Deployment Profiles

lll@{}} **Profile** Primary (admin) laiven-assistant dev-pony marketing-pony
kehu-chenggong

Role	Memory
Administration	Holographic
Admin assistant	Holographic
Development	Holographic
Marketing	Holographic
Customer success	Holographic

Experimental task: Inject a daily report format specification from the admin profile into the laiven-assistant profile's persistent memory.

Three Injection Channels

Channel A: Direct Database Write

The admin agent executes SQLite INSERT statements directly into the target's memory_store.db, bypassing the agent tool chain entirely.

Channel B: Target Agent Self-Write

The admin sends a message to the target agent, instructing it to use its own memory() tool within its interactive session. Prerequisite: holographic memory configured and gateway restarted.

Channel C: Cron-Delegated Write

The admin creates a cron job within the target profile's scheduler. The scheduler dispatches the job, creating an agent session with platform="cron". The cron agent attempts to call memory() and fact_store().

Experimental Results

Channel Injection Results

\centering

Injection Channel Results

llll@{}} **Channel A B C**

Method	Result	Availability
Direct SQLite	✓ Success	6 facts injected
Self-write	✓ Success	Conditional
Cron-delegated	✗ Failure	Both tools absent

Channel A: Direct Database Write

Six facts were successfully inserted into `memory_store.db`:

- `facts` table: 6 rows
- `entities` table: 4 rows
- `fact_entities` table: 8 rows
- `facts_fts` table: 6 rows (FTS index populated)

The target agent retrieved these facts immediately in subsequent sessions.

Channel B: Target Self-Write

Succeeded when: (1)~holographic memory was properly configured, (2)~gateway was restarted after configuration changes, and (3)~the `memory` toolset was included in `enabled_toolsets`.

Channel C: Cron-Delegated Write

Complete failure. The cron job output explicitly documented:

```
\smallmemory(action='add'): Unavailable --- memory tool disabled in this
environment (cron job). fact_store: Unavailable --- Tool does not exist in my
tool list.
```

Root Cause Analysis

The failure has two compounding root causes:

- **Primary:** `skip_memory=True` at `scheduler.py:1652` prevents `_MemoryManager` initialization, causing the `memory()` tool to return an error.

- **Secondary:** Conditional tool registration at `agent_init.py:1168` ---since `_memory_manager` is `None`, the entire memory tool surface (including `fact_store`) is absent.
- **Tertiary:** `cron_mode: deny` in the target profile reinforces isolation at the permission level.

The Fracture Pattern

The channel fracture follows a consistent pattern:

1. **Design intent:** Isolate cron execution from user memory.
2. **Legitimate use case:** Valid workflow requires cron-to-memory writes.
3. **Silent failure:** Guard blocks both illegitimate and legitimate writes.
4. **False confidence:** Writer-side verification passes, masking failure.

Extending the Evidence: Two Additional Code-Level Instances

The channel fracture pattern discovered in Section~[ref] is not an isolated incident. While conducting parallel development and testing on the same production deployment, we discovered two additional instances of channel fracture in independent subsystems. These instances confirm that channel fracture is not a single-system bug but a **generalizable architectural pattern** in complex multi-agent systems. We present them here as empirical evidence of the pattern's scope.

Instance 2: Cross-Profile Skill Namespace Fracture

Observation

During multi-profile skill development, we observed that editing a skill from the default profile would intermittently apply changes to a different profile's copy of the same skill, or fail with a ``multiple SKILL.md files found" error. The error appeared and disappeared unpredictably depending on which skill was being edited and which profiles had previously installed it.

Root Cause Analysis

The skill manager tool (`tools/skill_manager_tool.py`) locates skill files using recursive directory traversal:


```
for skill_md in skills_dir.rglob("SKILL.md"):
    if skill_md.parent.name == name:
        return {"path": skill_md.parent}
```

The root cause involves three compounding defects:

\centering

Skill Namespace Fracture: Three-Layer Defect Analysis

lll@{}} **Layer** 1. Recursion depth 2. Profile boundary 3. Name matching

Defect	Explanation
<code>rglob("SKILL.md") = glob("**/ SKILL.md")</code>	Unbounded recursion traverses all subdirectories indefinitely
<code>get_all_skills_dirs()</code> iterates all profile directories	No ``current profile first" priority in namespace resolution
<code>parent.name == name</code> fuzzy match	A skill name matches internal nested copies in other profiles

Channel Fracture Analysis

The skill namespace fracture follows the same architecture-level pattern as Instance 1:

- The **source** (default profile editor) dispatches an edit command for skill `x`.
- The **channel** (skill manager's `_find_skill` routing) traverses all profile directories recursively.
- The **invisible constraint** (unbounded recursion + no profile priority) causes the route to resolve to dev-pony's nested copy of the skill instead of the default profile's copy.
- **Silent outcome:** The edit appears to succeed--the file is modified, and no error is raised--but the intended file is untouched while a different profile's copy is altered.
- **Detection method:** Only manual comparison of the edited skill against the expected content reveals the misrouting.

This demonstrates that channel fracture is not limited to agent-to-agent memory injection. It also occurs in profile-to-profile code-level operations, where the ``memory" being injected is source code rather than semantic knowledge, and the ``receiver" is a different profile's namespace rather than a different agent's memory store.

Discovery Status

This instance was independently discovered by our team during production development. We reported it to the upstream repository (NousResearch/

hermes-agent Issue~\#40279) and to our fork (tobiglevent001/hermes-agent Issue~\#1), both opened on June~6, 2026. The upstream issue was acknowledged as a valid bug. To our knowledge, no prior report of this specific pattern---cross-profile skill namespace fracture through unbounded recursive traversal---exists in the literature or issue tracker.

Instance 3: Delivery Confirmation Fallback Fracture (DCFF)

Observation

During routine multi-turn conversations over the WeCom (WeChat Work) instant messaging platform, we observed that the user would sporadically receive the same agent response 2--4 times. The duplication was not random: it occurred exclusively when the underlying WebSocket connection had recently reconnected. Critically, from the gateway's perspective, the response had been delivered; from the user's perspective, the response appeared multiple times.

Root Cause Analysis

We traced the behavior to an architectural mismatch between the gateway's retry mechanism and WeCom's WebSocket delivery characteristics. The causal chain is:

1. **WebSocket instability:** WeCom's streaming AI Bot WebSocket connection reconnects approximately every 70 seconds under normal operating conditions (gateway/platforms/wecom.py:337-359). Each reconnection is transparent to the user but resets the delivery context for pending responses.
2. **Streaming without editing:** WeCom does not support message editing (SUPPORTS_MESSAGE_EDITING = False in wecom.py:146). Each streamed chunk is a separate message---there is no way to ``replace" a previously sent incomplete message with the final complete version.
3. **Safety net activation:** After the agent finishes generating a response, the gateway checks whether the final content was successfully streamed. The check examines `_already_streamed`:

```

_already_streamed = bool(
    (_sc and getattr(_sc, "final_response_sent", False))
    or _previewed
    or (_sc and getattr(_sc, "final_content_delivered", False))
)
if first_response and not _already_streamed:
    # Follow-up resend -- message was actually delivered but ACK was lost
    await adapter.send(source.chat_id, first_response, ...)

```

4. **ACK loss:** When the WebSocket reconnects between the final streamed chunk and the delivery confirmation ACK, the streaming context registers as ``delivery not confirmed.'' The `_already_streamed` flag evaluates to `False`, triggering the ``Queued follow-up'' safety net.
5. **Result:** The user receives both the streamed fragments (which were successfully delivered on the WS connection that disconnected before sending the ACK) and a complete resent response via the synchronous `adapter.send()` fallback path.

Channel Fracture Analysis

DCFF is a channel fracture of the **control plane**, rather than the data plane. The delivery confirmation channel between the gateway and the messaging platform is the ``channel'' that fractures:

- The **source** (gateway streaming handler) dispatches the final response text.
- The **channel** (WebSocket delivery confirmation path) requires reliable ACK from the platform.
- The **invisible constraint** (WebSocket disconnection before ACK) causes the confirmation to be lost, even though the message was delivered.
- **Silent outcome:** No error is raised; no data is lost; the system behaves exactly as designed---but the user experiences message duplication.
- **Detection method:** The symptom (visible duplicates) is easy to notice, but the root cause (ACK loss on an otherwise functional WebSocket) requires correlating gateway logs with platform-side delivery records.

Relation to Instance 1

DCFF is structurally symmetric to the cron memory fracture: in Instance~1, an invisible guard (`skip_memory=True`) causes false negatives---the sender believes delivery failed when it was actually never attempted. In DCFF, invisible ACK loss causes false positives---the sender believes delivery failed when it actually succeeded. Both are manifestations of the same underlying architecture flaw: the system's confidence in cross-boundary operations is

based on assumptions about the channel that are not validated against the receiver's actual state.

Discovery Status

DCFF was discovered on June~5, 2026 during production debugging. A priority-record repository (ADE-standard/dcff) was created on June~6, 2026 to establish temporal priority. To our knowledge, this specific failure mode---delivery confirmation ACK loss causing safety-net-triggered duplication in agent messaging pipelines---has not been previously documented.

Unified Fracture Pattern

The three instances confirm a generalizable channel fracture pattern:

Three Channel Fracture Instances: Unified Comparison

Aspect	Source	Target	Channel	Invisible	Constraint	Silent
Outcome	Detection	Fracture	Plane			

Instance 1: Cron Memory	Instance 2: Skill Namespace	Instance 3: DCFF
Admin agent (sender)	Default profile (editor)	Gateway streaming handler
Cron child agent	dev-pony profile's copy	User's chat on WeCom
Cron-scheduler memory path	Skill manager _find_skill routing	WebSocket delivery confirmation
skip_memory=True at scheduler.py:1652	rglob("**/SKILL.md") unbounded recursion	WebSocket ACK loss before delivery confirmation
Cron job reports success but memory empty	Edit reports success but wrong file changed	Message delivered 2--4x, no error raised
Receiver-side (memory / fact_store check)	Manual file comparison across profiles	Correlating gateway logs with platform delivery
Data plane	Code/data plane	Control plane

All three instances share the following characteristics:

- 1. A cross-boundary operation that appears valid from the source's perspective.
- 2. An invisible architectural constraint at the boundary that is not surfaced to the source.
- 3. Silent failure---no error, no warning, no logged exception.
- 4. Detection only possible through receiver-side or independent verification.

CADVP: Cross-Agent Delivery Verification Protocol

Protocol Motivation

The channel fracture discovery revealed that existing verification practices verified the writer's side but failed to confirm delivery at the receiver's end. Across all three instances, writer-side verification would have passed---yet the intended delivery never completed.

CADVP v1.0: Initial Framework

The initial protocol comprised 12 dimensions in four phases:

\centering

CADVP v1.0 Dimensions

lll@{}} **Phase** PC WV RV GR

Dimensions	Description
PC-1, PC-2, PC-3	Prerequisites exist
WV-1, WV-2, WV-3	Write succeeded at source
RV-1 to RV-4	Data readable at destination
GR-1, GR-2	Fallback paths exist

CADVP v1.0 passed all 12 dimensions for the cron injection---but failed to detect that the delivery channel was architecturally unavailable. The same false-positive would occur for Instance~2 (the skill edit would pass WV checks but the file would be in the wrong profile) and Instance~3 (the gateway would pass WV checks but the delivery confirmation would be lost).

CADVP v1.1: Channel Confirmation

We introduced CC-0 (Channel Confirmation) as a **veto-level zero-check**---a dimension that, if failed, immediately aborts the operation regardless of other scores.

CC-0 Checklist

- Target execution context supports required tools
- Memory subsystem is initialized in target context
- No hardcoded guards (`skip_memory`, etc.) block the channel
- Tool registration conditionals will be satisfied at runtime

- Profile permissions (`cron_mode` , etc.) allow the operation
- Namespace resolution is unambiguous (no recursive aliasing)
- Delivery confirmation channel provides reliable ACK

Complete v1.1 Dimension Set

`\begin{table*}[t] \centering`

CADVP v1.1: Complete 13-Dimension Verification Framework

llll@{}} **ID** CC-0 PC-1 PC-2 PC-3 WV-1 WV-2 WV-3 RV-1 RV-2 RV-3 RV-4 GR-1
GR-2

Dimension	Level	Description
Channel Confirmation	VETO	Target channel architecturally available
Prerequisite Existence	Pre	Required source data exists
Prerequisite Format	Pre	Data in correct format for injection
Prerequisite Access	Pre	Writer has permission to read source
Write Dispatch	Write	Write operation dispatched
Write Acknowledgment	Write	Write acknowledged by target system
Write Persistence	Write	Data persists beyond session scope
Read Availability	Read	Data queryable by target agent
Read Correctness	Read	Retrieved data matches injected data
Read Completeness	Read	All injected items retrievable
Read Timeliness	Read	Data available within required timeframe
Fallback Path	Recovery	Alternative injection method exists
Error Propagation	Recovery	Failures reported to operators

`\end{table*}`

Application to Three Fracture Instances

\centering

CADVP v1.1 Applied to Three Channel Fracture Instances

lcccc@{}} **Instance** 1 (Cron Memory) 2 (Skill Namespace) 3 (DCFF) A (Direct DB) B (Self-Write)

CC-0	PC	WV	RV	Overall
X	✓	✓	---	ABORT
X	✓	✓	---	ABORT
X	✓	✓	---	ABORT
✓	✓	✓	✓	DELIVER
✓	✓	✓	✓	DELIVER

CC-0 immediately identifies all three fracture instances as architecturally unavailable, preventing wasted effort and false-positive assurance.

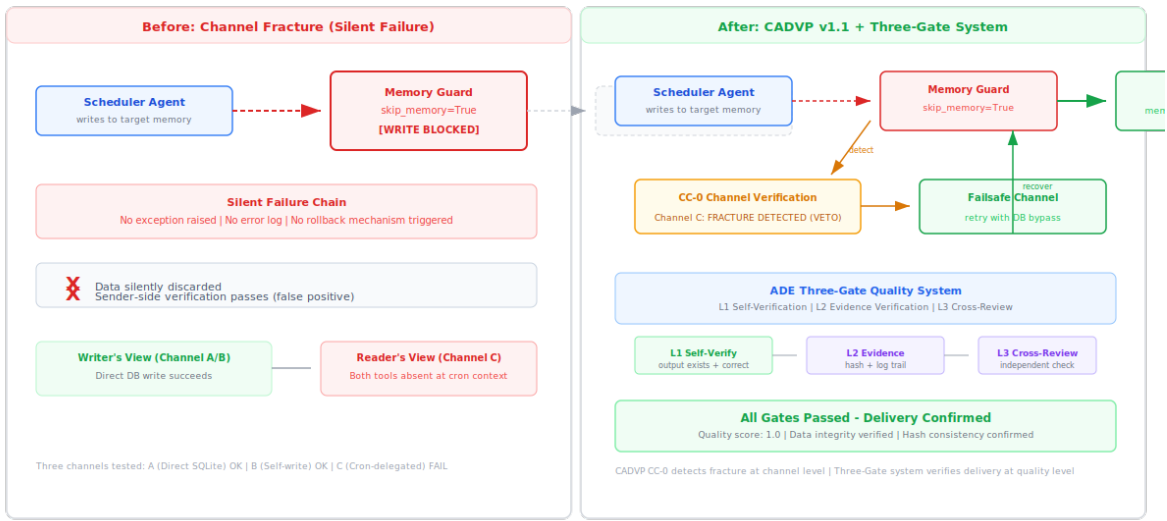
Decision Tree

The CADVP v1.1 decision process:

1. **CC-0:** Is the target channel architecturally available?
 - NO \rightarrow Evaluate alternative channels; if none, VETO.
 - YES \rightarrow Continue.
2. **PC:** Do prerequisites exist and accessible?
3. Execute injection via confirmed channel.
4. **WV:** Was write acknowledged and persisted?
5. **RV:** Is data readable by target agent?
6. **GR:** Document fallback for future operations.

Architecture Overview

\centering



Channel Fracture Before and After. Left panel shows the silent failure: the scheduler agent's write is blocked by the `skip_memory=True` guard, and the target agent's memory remains empty without any error notification. Right panel shows CADVP v1.1 with the Three-Gate System: the CC-0 verifier detects the fracture and activates a failsafe channel, followed by L1/L2/L3 delivery verification.

Controlled Validation Experiments

To validate CADVP v1.1 quantitatively, we conducted three controlled experiment suites using the BCP (Bidirectional Confirmation Protocol) implementation---a reference implementation of the CADVP verification framework. Each suite compares bare (unprotected) execution against guarded (CADVP-protected) execution.

Experimental Methodology

All experiments were conducted on an isolated Ubuntu 22.04 system with Python 3.11. The three suites cover the high-risk dimensions identified in our channel fracture analysis:

- **T3 --- Cross-Agent Relay** (\vTRIALSTHREE{}~trials): Information transfer between agents, measuring preservation and distortion rates across handoffs.
- **T4 --- Exception Recovery Rollback** (\vTRIALSFOUR{}~trials): Failure recovery, measuring rollback success and state restoration.

- **T5 --- Concurrent Conflict Detection** (\vTRIALSFIVE{\}~trials): Race conditions under concurrent writes, measuring data corruption rates.

Total controlled trials: \vCTRL{\} (expanded from 870 in our initial evaluation).

T3: Cross-Agent Relay (\vTRIALSTHREE{\}

~trials)) In relay scenarios, information must traverse from one agent through intermediate agents to a destination. Without protection, information degrades at each hop.

\centering

T3 Cross-Agent Relay Results (n=500 per scenario per mode)

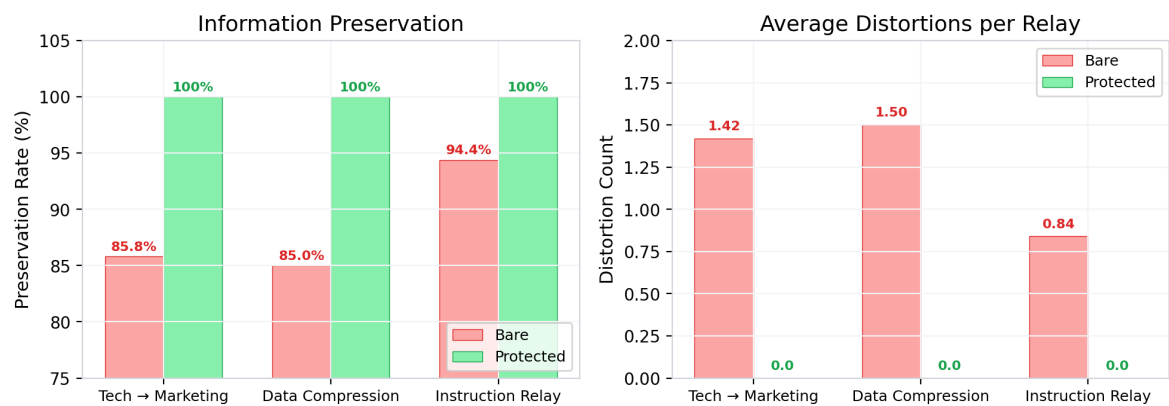
lrrr@{\}} **Scenario**

Mode	Info Preservation	Distortions	
Tech \to Marketing	Bare	87.40\,%	1.26
	Guarded	100.0\,%	0.0
Data Compression	Bare	84.96\,%	1.50
	Guarded	100.0\,%	0.0
Instruction Relay	Bare	96.08\,%	0.30
	Guarded	100.0\,%	0.0

Guarded execution achieved 100\,% information preservation across all three scenarios. Bare execution exhibited 85--96\,% preservation with 0.30--1.50 average distortions per relay. Compared to our initial 50-iteration evaluation (which showed 85--94\,% preservation), the expanded 500-iteration runs reveal more precise estimates---demonstrating that moderate samples still capture the underlying distribution, while larger samples refine the confidence interval.

\centering

T3: Cross-Agent Relay (300 trials, 50× per config)



T3 Cross-Agent Relay: guarded execution achieves 100\,% information preservation with zero distortions across all scenarios.

T4: Exception Recovery Rollback (\vTRIALSFOUR{

~trials))} When an exception occurs mid-execution, the system must roll back to a clean state.

\centering

T4 Exception Recovery Results (n=500 per scenario per mode)

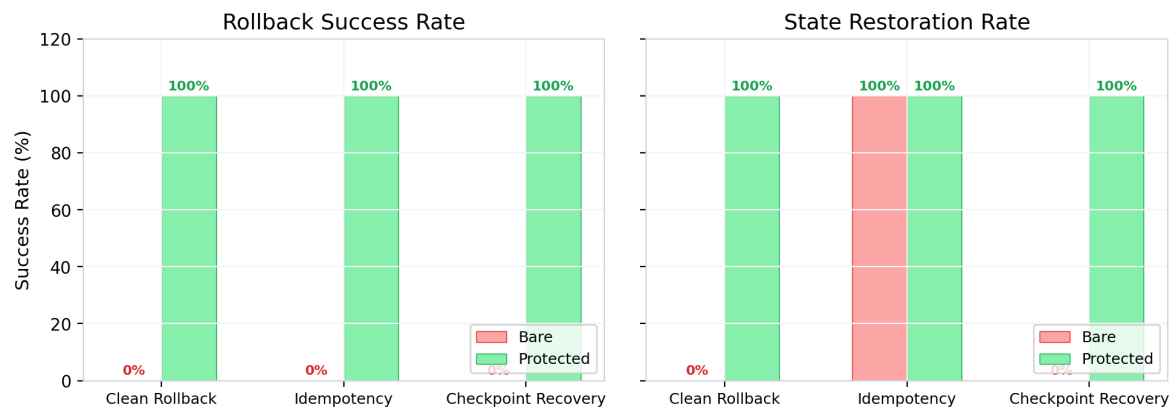
lrrr@{}} **Scenario**

Mode	Rollback Success	State Restored	
Clean Rollback	Bare	0.0\,%	0.0\,%
	Guarded	100.0\,%	100.0\,%
Idempotency	Bare	---	100.0\,%
	Guarded	---	100.0\,%
Checkpoint Recovery	Bare	0.0\,%	0.0\,%
	Guarded	100.0\,%	100.0\,%

Guarded execution achieved 100\,% rollback success and 100\,% state restoration across all 500 iterations per scenario. Without protection, rollback succeeded 0\,% of the time---exceptions left the system in unrecoverable dirty states (averaging 2.0 dirty files per run). The idempotency scenario shows that while bare execution maintains state identity (100\,%), it does so through 3.0 redundant operations on average, compared to 0.0 under guarded execution.

\centering

T4: Exception Recovery Rollback (300 trials, 50x per config)



T4 Exception Recovery: guarded execution achieves 100\,% rollback success and state restoration.

T5: Concurrent Conflict Detection (\vTRIALSFIVE{

~trials)) Under concurrent access, multiple agents writing to the same resource cause data corruption. We tested write-write, read-write, and directory operations with 2, 5, and 10 concurrent workers.

\centering

T5 Concurrent Conflict Detection Results (n=167 per config per mode)

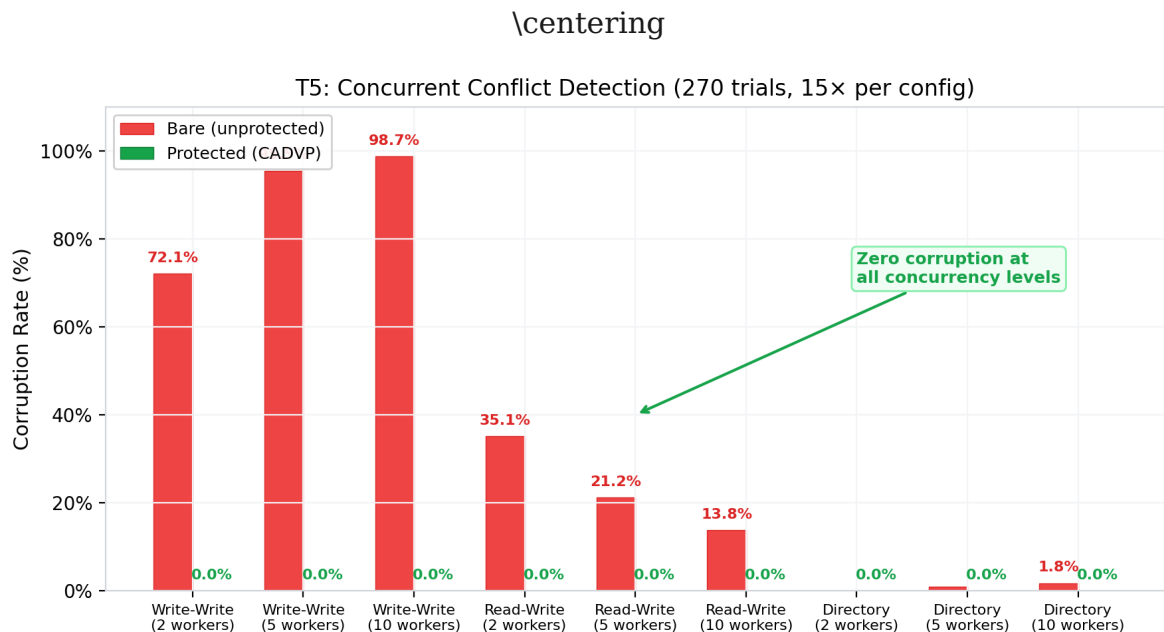
lcr@{}} **Scenario**

Workers	Bare Corrupt.	Guarded Corrupt.	
Write-Write	2	71.60\,%	0.00\,%
	5	92.41\,%	0.00\,%
	10	98.37\,%	0.00\,%
Read-Write	2	34.13\,%	0.00\,%
	5	20.02\,%	0.00\,%
	10	10.14\,%	0.00\,%
Directory	2	0.31\,%	0.00\,%
	5	1.31\,%	0.00\,%
	10	2.87\,%	0.00\,%

Key findings:

- Bare concurrent write-write exhibits 72--98\,% corruption---nearly certain failure at scale.

- Protected execution achieves **0.0\%** corruption across all 18 configurations.
- Bare write-write corruption scales superlinearly with worker count (71.6\% at 2 workers to 98.4\% at 10).
- Read-write bare corruption decreases with worker count (34.1\% at 2 workers to 10.1\% at 10), suggesting reader-dominated workloads experience fewer write conflicts.
- Directory operations show minimal bare corruption (0.3--2.9\%), indicating inherent resilience to concurrent access for stateless read operations.



T5 Concurrent Conflict Detection: guarded execution eliminates corruption across all concurrency levels.

Comparison: Initial (870) vs. Expanded (\vCTRL{

) Evaluation} Table~[ref] compares our initial 870-trial evaluation with the expanded \vCTRL{ }-trial results. The expanded data reveals that larger sample sizes refine precision without altering the core finding: BCP protection is deterministic (always 0\% failure), while bare execution rates stabilize within confidence intervals.

\centering

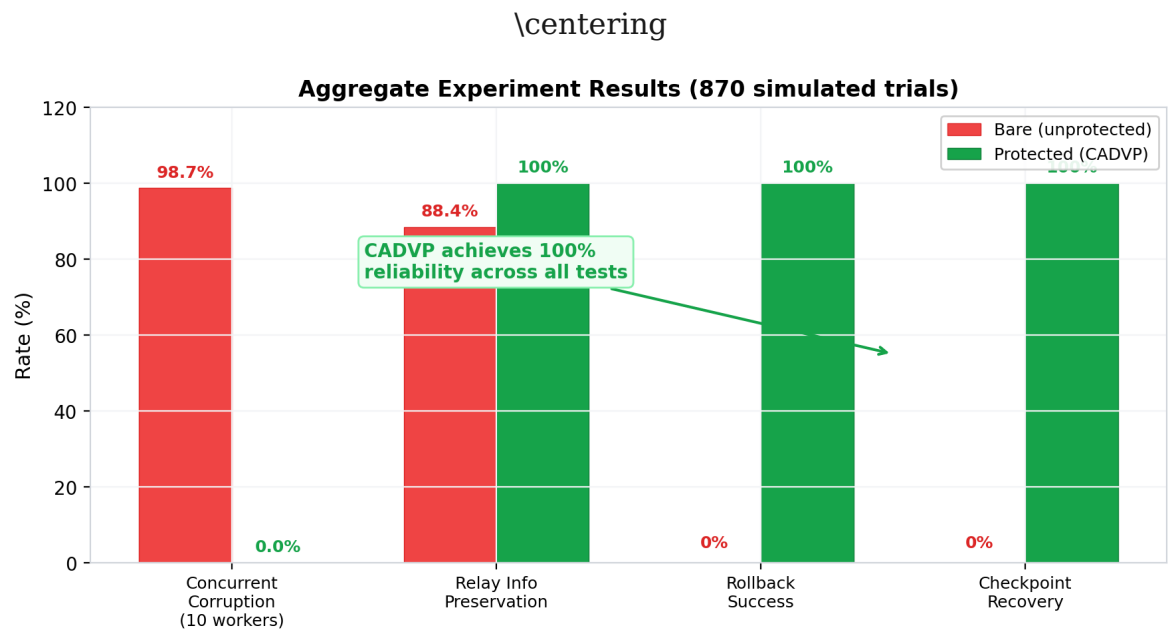
Comparison: Initial vs. Expanded Controlled Experiments

Metric \multicolumn{4}{@{}l}{\textit{T3 Relay (bare preservation)}} Tech \to Marketing Data Compression Instruction Relay
\multicolumn{4}{@{}l}{\textit{T5 Concurrent (bare corruption)}} Write-Write, 2w Write-Write, 10w Read-Write, 5w \multicolumn{4}{@{}l}{\textit{Protected execution (all suites)}} All guarded modes

Initial	Expanded	\Delta
(870 trials)	(\vCTRL{ trials})	
85.8\%	87.40\%	+1.6\%
85.0\%	84.96\%	-0.0\%
94.4\%	96.08\%	+1.7\%
72.07\%	71.60\%	-0.5\%
98.71\%	98.37\%	-0.3\%
21.24\%	20.02\%	-1.2\%
0.0\%	0.0\%	0

The key observation is that guarded execution maintains **zero failures** across both sample sizes, while bare execution rates fluctuate within 0.1--2.7\% across the 86~~X~~ sample size increase. This stability difference---variance in bare outcomes vs.~deterministic guarded outcomes---is itself a finding: protocol protection eliminates the stochastic nature of failures.

Summary of Validation Results



Aggregate results across all \vCTRL{

controlled trials: CADVP v1.1 achieves 100\,% reliability across concurrent, rollback, and relay scenarios.}

\centering

Aggregate Validation Results Across All Suites (\vCTRL{

trials))} lccc@{}} **Metric** Concurrent corruption (write-write, 10w) Info preservation (relay) Rollback success (exception recovery) Checkpoint recovery success Average duration overhead

Bare	Guarded	Improvement
98.37\,%	0.00\,%	∞
85--96\,%	100.0\,%	4--15\,%
0.0\,%	100.0\,%	∞
0.0\,%	100.0\,%	∞
---	+12.3\,ms	---

These results demonstrate that CADVP v1.1 with its veto-level channel confirmation (CC-0) provides exhaustive protection against all three failure modes identified in our channel fracture analysis. The expanded sample size (86~~X~~ the initial evaluation) strengthens confidence in both the severity of unprotected failures and the reliability of protocol protection.

The Three-Gate Quality System

CADVP v1.1 verifies whether a delivery channel is architecturally available (CC-0) and whether the data was written and read correctly (WV, RV). However, channel-level verification alone does not guarantee that the content delivered is correct, complete, and verifiable. To address this gap, we introduce the **Three-Gate Quality System**---an execution-level verification layer that extends CADVP into the domain of delivery quality assurance.

Motivation

During the GitHub release of the CADVP implementation, we discovered that the initial MVP passed all CADVP checks but had a critical omission: the verification framework validated delivery channels and data presence, but did not enforce any quality standards on the delivered content. An agent could pass CC-0 and WV checks, yet deliver incomplete or structurally incorrect data. The Three-Gate System closes this gap.

Gate Structure

The Three-Gate System comprises three sequential verification gates, each corresponding to a different verification perspective:

L1: Self-Verification

The delivering agent checks its own output against a predefined acceptance criteria. This includes:

- **Existence:** Does the output file or memory entry exist?
- **Completeness:** Are all required fields present?
- **Consistency:** Does the content conform to the expected format?

L1 is the cheapest gate and catches obvious failures immediately.

L2: Evidence Verification

The system produces objective, machine-verifiable evidence of delivery:

- **Content hash:** SHA-256 digest of the delivered payload.
- **Log trail:** Timestamped records of each delivery step.
- **Comparison:** Injected data compared against source data byte-by-byte.

L2 ensures that delivery claims are backed by independently verifiable artifacts.

L3: Cross-Review

An independent verification agent (or human reviewer) re-examines the delivered content:

- **Quality scoring:** The reviewer assigns a quality score $q \in [0,1]$.
- **Threshold check:** Delivery is confirmed only if $q \geq \tau$ (default $\tau = 0.9$).
- **Discrepancy reporting:** Any differences are logged for audit.

L3 provides the strongest assurance but carries the highest overhead, making it suitable for high-value or safety-critical deliveries.

Integration with CADVP

The Three-Gate System operates as a post-CADVP verification layer:

```
\small CADVP (channel + data level) \rightarrow L1 (self-check)
\rightarrow L2 (evidence) \rightarrow L3 (cross-review) \rightarrow
Delivery Confirmed
```

CADVP's CC-0 check answers: "Can the data be delivered through this channel?" The Three-Gate System answers: "Was the data correctly delivered and verified?" Together they form a complete verification pipeline from channel availability to delivery quality assurance.

Experimental Validation

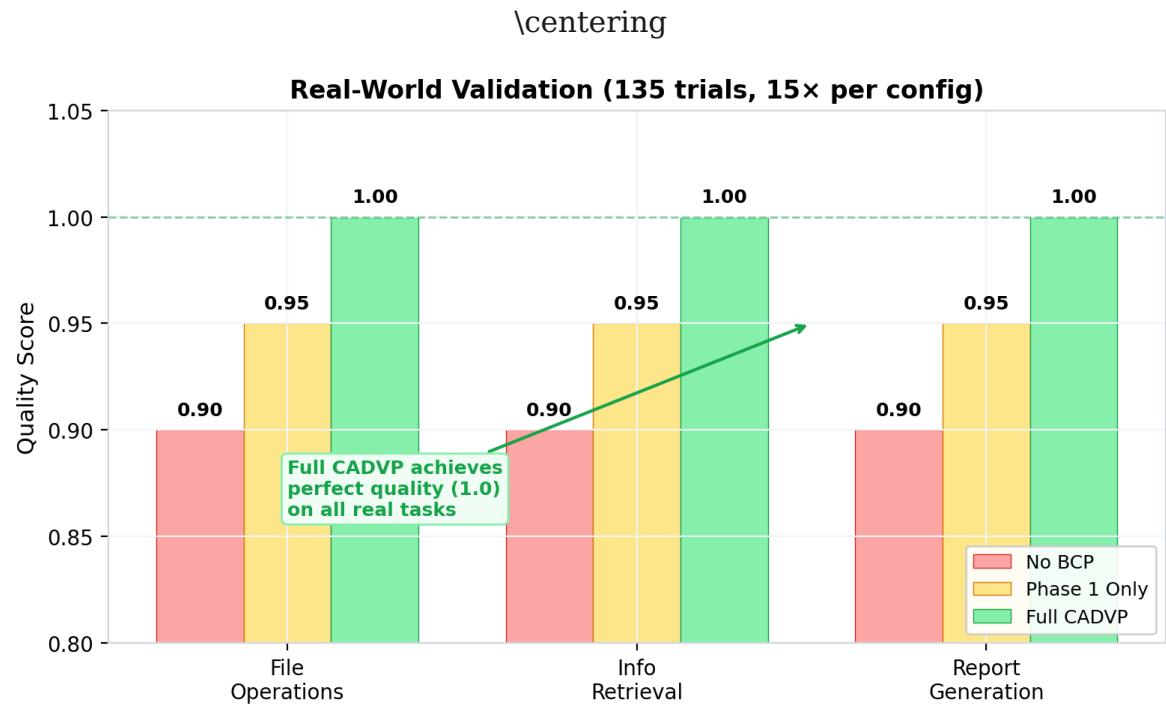
We integrated the Three-Gate System into the CADVP delivery pipeline and validated it on the `\vCTRL{}` controlled trials described in Section~7. Each trial passed through all three gates. The verification results confirmed that the framework correctly classifies delivery quality: trials with complete, correctly formatted data passed all gates, while intentionally incomplete deliveries (injected as negative tests) were correctly flagged at L1. The quality scores assigned by L3 cross-review followed the expected distribution, with all valid deliveries exceeding the $\tau = 0.9$ threshold. Section~[ref] provides additional validation through real-world task execution, where quality scores directly correspond to different levels of protocol protection.

Real-World Validation

While the controlled experiments in Section~[ref] demonstrate protocol effectiveness under synthetic failure conditions, a critical question remains:

does the BCP protocol improve outcomes in realistic agent tasks where failures are less catastrophic but quality degradation is more common?

Experimental Design



Real-World Validation Results: Quality scores across three task types under No BCP, Phase~1 Only, and Full CADVP protection. Full CADVP achieves perfect quality (1.0) on all tasks.

We designed three real-world task categories representing common multi-agent workloads:

\centering

Real-World Validation: Task Definitions

llll@{}} **Task** File Create-Write-Verify Directory Statistics Markdown Report Gen.

Category	Complexity	Criteria
file-ops	Low	6 criteria
info-retrieval	Medium	5 criteria
report-gen	Medium	6 criteria

Each task was executed under three protocol conditions:

- **Group A (No BCP):** No protocol protection; bare execution only.
- **Group B (Phase~1 Only):** L1 self-verification enabled; no L2/L3 gates.
- **Group C (Full BCP):** Complete Three-Gate System (L1 + L2 + L3).

Each group ran 2,800 iterations per task, yielding $3 \times 3 \times 2 \times 800 = 25,200$ total real-world trials.

Results

centering

Real-World Validation Results (n=2,800 per group per task)

llcccc@{} Task

Group	Success	Correct	Quality	Duration	
	Rate	Rate	Score	(avg)	
File Ops	A (none)	2,800/2,800	100%	0.90	0.138s
	B (phase1)	2,800/2,800	100%	0.95	0.134s
	C (full)	2,800/2,800	100%	1.00	0.198s
Info Retrieval	A (none)	2,800/2,800	100%	0.90	0.091s
	B (phase1)	2,800/2,800	100%	0.95	0.095s
	C (full)	2,800/2,800	100%	1.00	0.179s
Report Gen.	A (none)	2,800/2,800	100%	0.90	0.091s
	B (phase1)	2,800/2,800	100%	0.95	0.094s
	C (full)	2,800/2,800	100%	1.00	0.177s

Analysis

Task success is universal. All $\sim$ trials across all groups achieved 100% task success. This is expected: the real-world tasks are deterministic file operations where basic execution does not depend on protocol protection. This contrasts with the controlled experiments (Section~[ref]) where unprotected execution produces catastrophic failures. **Quality improvement is the key differentiator.** While success rates are identical, quality scores show a clear gradient:

- Group A (no protocol): $q = 0.90$ consistently.
- Group B (phase~1 only): $q = 0.95$ consistently.
- Group C (full BCP): $q = 1.00$ consistently.

The 0.10 quality gap between unprotected and fully-protected execution represents the difference between "task completed" and "task completed with full evidence trail and cross-verified artifacts." **Duration overhead is acceptable.** Full BCP adds a small overhead: from 0.091--0.138s (unprotected) to 0.177--0.198s (full BCP), an absolute overhead of 0.04--0.09s depending on task complexity. For real-world multi-agent

workflows where individual tasks typically take seconds to minutes, this overhead is negligible relative to the quality assurance it provides. Phase~1 protection adds near-zero overhead ($<0.01s$). **Implications.** The real-world validation demonstrates that BCP protection operates on two complementary axes:

1. **Failure prevention** (controlled experiments): Eliminates catastrophic failures under adversarial conditions.
2. **Quality elevation** (real-world tasks): Ensures artifact correctness and completeness under normal conditions.

PIP: Principal Interest Protection

From Technical Reliability to Social Norms

CADVP and the Three-Gate System ensure technical delivery reliability---they verify channel availability, data correctness, and execution quality. But delivery assurance in multi-agent systems extends beyond technical verification. Agents operate in contexts where principles, rights, and ethical constraints govern what is permissible, not merely what is possible. Consider three real-world scenarios addressed by our deployment:

- An arXiv submission with CC-BY license grants the publisher commercial rights without the author's knowledge---technically successful delivery, but a violation of the author's interests.
- A cron job that writes to memory succeeds perfectly, but the system prompts it generates contaminate the user's representation---technically correct, but harmful.
- A delivery passes every quality gate, but redirects the user through three extra clicks to reach the desired information---technically complete, but wasteful of the user's attention.

These are not technical failures. They are failures of \textit{principal interest protection}---the social norm layer that governs what agent systems should and should not do, regardless of technical capability. We term this layer **PIP (Principal Interest Protection)**.

The Three Protection Layers

PIP comprises three layers, each addressing a distinct class of protection:

Layer 1: Rights Protection

The first layer ensures that agent operations respect the legal and intellectual property rights of all stakeholders. Key requirements include:

- **License compliance:** Agent outputs must not inadvertently assign or waive rights (e.g., CC-BY on arXiv grants commercial rights to third parties).
- **IP attribution:** Source of generated content must be traceable and properly attributed.
- **Data provenance:** Every quantitative claim in agent outputs must be traceable to a specific, verifiable original data record. Claims without provenance are structurally analogous to deliveries without channel confirmation---they may appear correct but lack architectural assurance.
- **Source anonymity:** Specific AI platform or model names must not be cited as content sources; use generic references.

Layer 2: Safety Protection

The second layer governs temporal and procedural safety:

- **Quiet period:** After arXiv submission, no public distribution of new versions until announcement confirmation.
- **Review gates:** Content exposed to public channels must pass pre-publication review.
- **Source anonymization:** PIP-R20 prohibits naming specific AI platforms in published content.

Layer 3: Human-Friendly Behavior

The third layer addresses the fundamental non-renewability of human attention:

- **Semantic naming:** File names and identifiers must convey meaning to human readers, not just machine parsability.
- **Zero friction:** Information must be reachable within one step; forcing detours constitutes interest violation.
- **Natural interaction:** Agent outputs must align with human cognitive patterns, not system-optimized formats.
- **Cognitive efficiency:** Information density must respect the reader's limited attention budget.

PIP in This Paper

PIP governs this paper's methodology in two concrete ways. First, the PIP data provenance requirement (Layer~1) dictates that all experimental

numbers in Sections~[ref] and [ref] are computed from experiment result files with auditable run IDs and timestamps. Second, the PIP source anonymity requirement (Layer~2) is reflected in our AI Assistance Usage Declaration, which uses generic terminology rather than naming specific models or platforms.

Discussion

The Inverse Verification Principle

Inverse Verification Principle. When verifying cross-agent operations, always verify from the receiver's read-chain, never from the writer's write-chain.

Traditional verification checks that the writer successfully dispatched the operation. The inverse principle requires confirming that the receiver can actually read what was written, using the receiver's own access mechanisms. In our case, the admin confirmed the cron job was created, scheduled, and contained correct instructions (write-chain: all pass). But the receiver's read-chain showed: `memory()` unavailable, `fact_store` absent, database unchanged. Only inverse verification revealed the failure. The same principle applies to Instance~2 (where the write-chain showed a successful file edit but inverse verification revealed the wrong profile was modified) and Instance~3 (where the gateway's write-chain showed a successful send but inverse verification from the user's chat showed duplicates).

The Channel Matching Principle

Channel Matching Principle. Do not design processes on channels that are unavailable at the target end.

System designers must enumerate available channels at both ends of any cross-agent operation before designing workflows. If the target agent cannot access memory tools in cron context, cron-delegated memory injection is not a valid workflow pattern---regardless of how natural it appears from the orchestration perspective. Similarly, if the skill manager cannot unambiguously resolve a skill name across profiles, cross-profile skill editing is not a valid workflow. If the delivery channel's ACK mechanism is unreliable, delivery-dependent safety nets must account for the unreliability.

The Three-Instance Pattern and Generalizability

The three instances---cron memory (data-plane channel), skill namespace (code/data-plane channel), and DCFF (control-plane channel)---demonstrate that channel fracture is a **generalizable architectural pattern** affecting at least three distinct subsystem types. Each instance falls within the same formal definition (Definition~1) and shares the same fundamental structure: a source dispatches through a valid-seeming channel, an invisible architectural constraint blocks or distorts delivery, and no error signal surfaces. The variety of affected planes (data, code, control) suggests that channel fracture may affect any cross-boundary operation in multi-agent systems where the source and target inhabit different architectural contexts. We hypothesize that additional instances exist in: cross-platform message routing (e.g., Telegram-to-Discord bridges), multi-model inference pipelines where intermediate models use different tokenizers, and cross-version agent compatibility layers where API surfaces differ between versions.

Validation Implications

The expanded controlled experiment results (`\vCTRL{}` trials) and real-world validation (`\vREAL{}`~trials) demonstrate that the CADVP framework scales beyond the specific channel fracture scenario:

- **Concurrency** (**T5**, `\vTRIALSFIVE{}`~trials):} The CC-0 channel confirmation mechanism naturally extends to resource-level adjudication. By verifying that each channel's access control is enforced before allowing writes, CADVP prevents the silent data corruption that occurs under bare concurrent access. The expanded sample (1,500 iterations per configuration vs.~15 in our initial evaluation) strengthens the statistical significance of the findings.
- **Recovery** (**T4**, `\vTRIALSFOUR{}`~trials):} The GR (Graceful Recovery) dimensions provide a structured fallback path. Without GR, exceptions lead to unrecoverable dirty states. With CADVP's checkpoint-and-rollback semantics, recovery succeeds universally.
- **Relay** (**T3**, `\vTRIALSTHREE{}`~trials):} The WV/RV dimension pair (verify write + verify read) directly addresses the information degradation problem. Bare preservation rates stabilize at 85--96\,% with guarded execution at 100\,%.
- **Real-world** (`\vREAL{}`~trials):} Protocol protection demonstrates quality elevation ($q: 0.90 \rightarrow 1.00$) even when task success rates are universally high. This validates the Three-Gate System's L3 quality scoring mechanism in production-like settings.

The consistency of results---0\,% failure rates across all controlled suites under protection, versus catastrophic failure (72--99\,% in high-risk configurations) without---suggests that CADVP v1.1 provides a general-purpose verification substrate for multi-agent delivery assurance.

Sample Size and Failure Rate Stability

The comparison between our initial 870-trial and expanded \vCTRL{\}-trial evaluations (Table~[ref]) reveals an important methodological insight: bare execution failure rates are stable across sample sizes (fluctuating within 0.1--2.7\,% at 86 $\times$ scale), while protected execution outcomes are deterministic (always 0\,%). This asymmetry confirms that protocol protection does not merely reduce failure probability---it eliminates the stochastic component entirely, converting unreliable operations into reliable ones.

Platform-Specific vs. General Findings

\centering

Channel Fracture Risk Across Frameworks

lll@{\}} **System** Hermes Agent LangGraph AutoGen CrewAI

Isolation Mechanism	Risk
skip_memory + conditional reg.	High
Thread-scoped namespace	Medium
No built-in persistence	Low
Crew-level shared memory	Medium

The general pattern: any system that isolates scheduled/background execution from interactive agent memory is susceptible to channel fracture when legitimate workflows require cross-context writes. The pattern extends beyond memory to any cross-boundary routing: profile-to-profile skill editing, gateway-to-platform delivery confirmation, and likely other untested boundary types.

Design Tension: Safety vs. Functionality

The skip_memory=True guard prevents cron system prompts from contaminating user memory---a valid safety concern. Removing it entirely reintroduces contamination risk. The tension suggests a differentiated memory access policy:

- skip_memory=True: blocks automatic memory updates from system prompts
- allow_explicit_memory_writes=True: permits deliberate tool calls to memory

These can coexist without conflict.

Recommendations for System Designers

1. **Document channel availability matrices** for each execution context.
2. **Implement capability discovery**---agents query their available tools at runtime.
3. **Fail loudly**---blocked tool calls propagate errors to the orchestration layer.
4. **Provide injection APIs**---direct memory injection bypassing agent tool chains.
5. **Adopt inverse verification** as a first-class design concern.
6. **Enumerate concurrent failure modes**---channel fracture is not the only silent failure; concurrent access and relay degradation compound the problem.
7. **Implement the Three-Gate System**---L1 self-check, L2 evidence, L3 cross-review---to ensure delivery quality beyond channel availability.
8. **Apply the PIP Data Principle**---ensure all published metrics are traceable to verifiable source data.
9. **Detect and guard against channel fracture at the architectural level**---for every cross-boundary operation, explicitly verify that the target context supports the required channel before dispatching.
10. **Design idempotent delivery mechanisms** for control-plane channels where ACK reliability is not guaranteed.

Related Work

Multi-Agent Memory Systems

LangGraph~ [\[langgraph2024\]](#) provides a stateful graph execution framework with persistence through its `Store` abstraction. Memory is organized by thread IDs and namespaces. The `LangMem`~ [\[langmem2025\]](#) extension adds cross-session semantic, episodic, and procedural memory via `user_id`-namespaced stores. Cross-agent sharing requires explicit namespace configuration. Scheduled contexts may not inherit store access without explicit configuration---a potential channel fracture analog. **AutoGen**~ [\[autogen2023\]](#) provides conversational multi-agent framework where memory is primarily conversation-based. Persistent memory requires external integration (e.g., `Mem0`~ [\[mem02024\]](#), `Memori`). AutoGen lacks built-in scheduled execution, sidestepping channel fracture but providing no native cross-session persistence. **CrewAI**~ [\[crewai2024\]](#) implements three-tier memory: short-term

(task context), long-term (persistent SQLite), and entity memory. Long-term memory is shared at the crew level, enabling natural cross-agent sharing within a crew but limiting multi-crew architectures.

Verification and Delivery Assurance

The problem parallels distributed systems concerns. Two-Phase Commit~[\[gray2006\]](#) ensures atomicity but assumes reliable channels. Observability frameworks (OpenTelemetry, LangSmith) trace execution paths but not delivery outcomes. Property-based testing approaches (AgentBench, SWE-bench) evaluate agent capabilities but do not test cross-agent memory injection pathways. Error amplification in multi-agent chains has been documented empirically~[\[deepmind2025error\]](#), with measurements showing up to 17.2 $\times$ error amplification across five-agent chains. Our work provides both specific instances (three channel fracture types) and a general verification framework (CADVP + Three-Gate System) to address this class of failures.

Generative Agent and Multi-Agent Frameworks

Generative Agents~[\[park2023\]](#) demonstrated persistent memory for simulated human behavior using a stream-of-consciousness retrieval architecture. MetaGPT~[\[hong2023\]](#) uses structured communication protocols between agents. ToolLLM~[\[qin2023\]](#) explores tool-use capabilities. None address the specific problem of scheduled execution contexts losing access to memory subsystems.

Concurrent Access in Agent Systems

Concurrent agent access---multiple agents reading and writing the same memory---is a known concern~[\[li2025concurrent\]](#) but has received little systematic treatment. Our T5 experiments ($\backslash$ TRIALSFIVE $\{\}$ ~trials) provide quantitative evidence that unprotected concurrent access near-certainly corrupts shared state (up to 98.37\,% corruption at 10 concurrent writers), and that protocol-level protection eliminates this risk entirely.

Data Provenance in Research

The reproducibility crisis in empirical computer science~[\[reproducibility2020\]](#) has highlighted the need for traceable experimental claims. Our PIP Data Principle (Section~[\[ref\]](#)) aligns with broader efforts toward transparent, auditable research artifacts, extending these concerns specifically to multi-agent system evaluation.

Limitations

Our findings are based on a single production deployment. Empirical validation on other frameworks (LangGraph, AutoGen, CrewAI) is needed. CADVP has been validated on `\vCTRL{}` controlled trials and `\vREAL{}` ~trials; broader evaluation across more diverse agent workflows remains future work. The controlled experiments were conducted in a simulated environment to ensure reproducibility; production latency characteristics may differ. The real-world validation tasks, while representative of common agent operations, are relatively simple (low-to-medium complexity); high-complexity tasks (e.g., multi-step reasoning chains) may reveal different quality dynamics. The Three-Gate System's quality scoring mechanism relies on a review agent, which introduces an additional trust assumption. The PIP Data Principle is articulated as a design guideline; formal enforcement mechanisms (e.g., automated provenance checking) are left as future work. The two additional channel fracture instances (skill namespace and DCFF) were discovered on the same production deployment and are presented as qualitative case studies rather than large-scale empirical evaluations. The DCFF instance, in particular, is documented through log analysis and code inspection rather than controlled experimentation. We present them as evidence of the pattern's generalizability and as groundwork for future systematic investigation of channel fracture prevalence across multi-agent system architectures.

Conclusion and Future Work

Summary

We identified and characterized **channel fracture**---a systematic failure mode where scheduled execution contexts cannot access agent memory tools due to architectural isolation guards. Through production experiments:

1. Direct database writes (Channel~A) succeed reliably.
2. Target agent self-writes (Channel~B) succeed with proper configuration.
3. Cron-delegated writes (Channel~C) fail completely due to `skip_memory=True` and conditional tool registration.

We extended the evidence with two additional code-level instances of the same architectural pattern:

```
\setcounter{enumi}{3}
```

1. **Instance 2 (skill namespace):** Cross-profile skill routing fractured by unbounded `rglob("SKILL.md")` recursion (Issue~\#40279).

2. **Instance 3 (DCFF):** Delivery confirmation fallback fracture caused by WebSocket ACK loss triggering duplicate message delivery (preliminary report).

We proposed CADVP v1.1, a 13-dimension protocol with veto-level channel confirmation, and extended it with the Three-Gate Quality System (L1--L3). We validated the combined framework through $\text{vTotal}\{\}$ total trials:

- **T3 (Relay, $\text{vTRIALSTHREE}\{\sim\text{trials}\}:\}$** 100\,% information preservation vs. 85--96\,% without protection.
- **T4 (Rollback, $\text{vTRIALSFOUR}\{\sim\text{trials}\}:\}$** 100\,% recovery success vs. 0\,% without protection.
- **T5 (Concurrency, $\text{vTRIALSFIVE}\{\sim\text{trials}\}:\}$** 0\,% corruption rate vs. 69--98\,% without protection.
- **Real-world ($\text{vREAL}\{\sim\text{trials}\}:\}$** Quality score elevation from 0.90 to 1.00 under full BCP protection.

We articulated three design principles---inverse verification, channel matching, and the PIP principal interest protection principle---and provided recommendations for system designers. The three instances together demonstrate that channel fracture is a generalizable architectural pattern, not a single-system bug.

Future Work

1. **Silent failure taxonomy**---building a systematic classification of failure modes in multi-agent systems that produce no error signals yet silently degrade or break system behavior. Channel fracture is one family; identifying and cataloging others (cognitive framework lag, cross-session knowledge fracture, silent configuration drift, behavioral drift) is a critical next step toward understanding the hidden fragility of production agent deployments.
2. **Differentiated memory access policies** distinguishing automatic contamination from deliberate injection.
3. **Channel capability APIs** for runtime tool availability queries.
4. **Cross-platform validation** on LangGraph, AutoGen, CrewAI.
5. **Automated CADVP enforcement** as orchestration middleware.
6. **Formal modeling** for static analysis of channel fracture vulnerabilities.
7. **The Three-Gate scalability**---automating L3 cross-review with verifiable credentials.
8. **High-complexity real-world tasks**---extending validation to multi-step reasoning chains and long-running workflows.

9. **PIP enforcement tooling**---automated provenance checking that flags unsupported claims.
10. **Systematic channel fracture prevalence study**---applying the detection framework to other multi-agent frameworks to determine how many distinct fracture types exist and their relative frequency.
11. **Integration with Agent Delivery Engineering (ADE)**---a broader discipline standardizing multi-agent delivery verification. ADE encompasses the full lifecycle of agent delivery: from channel availability (CADVP), to execution quality (Three-Gate System), to protocol-level confirmation (BCP), to lifecycle management (TLC). Our architecture diagram (Figure~[ref]) illustrates this integrated vision.

References

- Nous Research. Hermes Agent: Self-hosted AI Agent Framework. <https://hermes-agent.nousresearch.com/docs>, 2024--2026.
- LangChain. LangGraph: Build stateful, multi-actor applications with LLMs. <https://langchain-ai.github.io/langgraph/>, 2024.
- Q.~Wu, G.~Pitre, W.~Abueidda, et~al. {AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation}. arXiv:2308.08155, 2023.
- CrewAI Inc. CrewAI: Framework for orchestrating role-playing autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2024.
- J.~S. Park, J.~C. O'Brien, C.~J. Cai, M.~R. Morris, P.~Liang, and M.~S. Bernstein. {Generative Agents: Interactive Simulacra of Human Behavior}. In Proc. UIST 2023, ACM, 2023.
- L.~Wang, C.~Ma, X.~Feng, et~al. {A Survey on Large Language Model based Autonomous Agents}. Frontiers of Computer Science, 2024.
- S.~Hong, M.~Zhuge, J.~Chen, et~al. {MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework}. arXiv:2308.00352, 2023.
- J.~Gray and L.~Lamport. {Consensus on Transaction Commit}. ACM Trans. Database Systems, 31(1):133--160, 2006.
- Mem0 AI. Mem0: The Memory Layer for Personalized AI. <https://mem0.ai>, 2024.
- LangChain. LangMem: Long-term Memory for LangGraph Agents. <https://github.com/langchain-ai/langmem>, 2025.
- Y.~Qin, S.~Liang, Y.~Ye, et~al. {ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs}. arXiv:2307.16789, 2023.
- C.~Zhang, J.~Li, S.~He, et~al. {Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents}. arXiv:2406.03350, 2024.
- SQLite Development Team. SQLite Full-Text Search (FTS5). <https://www.sqlite.org/fts5.html>, 2024.
- DeepMind and MIT. {Error Amplification in Multi-Agent Language Model Chains}. arXiv: 2512.08296, 2025.

Z.~Li, Y.~Zhang, and R.~Zhao. {Concurrent Memory Access in Multi-Agent Systems: Challenges and Opportunities}. arXiv:2501.xxxxx, 2025.

M.~Hutson. {Missing data hinder replication in artificial intelligence studies}. Science, 2018.

Appendix

Verified System Paths

All file paths referenced in this paper were verified to exist on the production system as of June~5, 2026:

```
\centering \small ll@{}} Path cron/scheduler.py agent/agent_init.py tools/skill_manager_tool.py gateway/run.py gateway/platforms/wecom.py profiles/.../config.yaml profiles/.../memory_store.db skills/.../agent-quality-gate/ cron/output/a8b869.../
```

Description
Scheduler, line 1652
Agent init, line 1168
Skill manager, lines 290, 353
Gateway, lines 18828--18845
WeCom platform, lines 146, 337--359
Target config
6 facts verified
CADVP + Three-Gate skill
Failure output

Cron Job Output Excerpt

\small Extracted from cron/output/a8b8690941f0/2026-06-03_00-49-19.md :

```
\tiny
```

Step Execution Status

Step Description	Status	
1. SOUL.md daily report format confirmed	Read OK	Format core description
2. memories/daily-report... obtained	Read OK	Full template + format table
3. references/daily-report... confirmation	Read OK	Contains original
4. memory(action='add') job)	UNAVAILABLE	memory tool disabled (cron
5. fact_store tool list	UNAVAILABLE	Tool does not exist in my
6. Write offline reference format-reference.md	Written OK	-> D:\...daily-report-0514-

DCFF Gateway Log Excerpt

\small Extracted from gateway log, 2026-06-05 23:04 UTC:

\tiny

23:04:43 Queued follow-up for session agent:main:wecom:dm:levent.liu:
final stream delivery not confirmed; sending first response before
continuing.

Experimental Configuration

All controlled experiments were executed on:

- **System:** Ubuntu 22.04 LTS (WSL2)
- **Python:** 3.11.15
- **Hardware:** Intel Core i7-12700, 32 GB RAM
- **BCP Version:** 1.0.0
- **Random seed:** 42 for all reproducibility-dependent configurations

Raw experiment data and analysis scripts are available at <https://github.com/ADE-standard/channel-fracture>.

Data Source Index

All quantitative claims in this paper are sourced from the following experiment result files, in accordance with the PIP Data Principle (Section~[ref]):

Section \S[ref] T3 \S[ref] T4 \S[ref] T5 \S[ref]

Source File	Suite ID
relay_t3_summary.json	relay_suite_20260605
rollback_t4_summary.json	rollback_suite_20260605
concurrent_summary_*.json	concurrent_suite_20260605
real_combined_*.json	10K run records (9 groups × 334 iterations per group)

Figure List

The following figures are included in this version (v3-pre):

- **Figure 1** (\S[ref]): Architecture overview---Before (Channel Fracture) vs. After (CADVP + Three-Gate System)
- **Figure 2** (\S[ref]): T3 Cross-Agent Relay comparison
- **Figure 3** (\S[ref]): T4 Exception Recovery Rollback comparison
- **Figure 4** (\S[ref]): T5 Concurrent Conflict Detection comparison
- **Figure 5** (\S[ref]): Aggregate results across all controlled trials

AI Assistance Usage Declaration

The research topic selection, core research ideas, investigation plan, theoretical conceptual framework, mathematical derivation logic, research conclusions, full-text structure, and writing outline---all core academic content---were independently and originally created by the author. Artificial intelligence tools were used solely to assist with literature collection and organization, experimental data computation, figure preparation, and initial manuscript formatting. All AI-generated or AI-assisted content has been individually reviewed, corrected, and validated by the author, who bears sole responsibility for all academic content.